

Container und Virtuelle Maschinen – Isolationsmechanismen, Bedrohungsmodelle und Sicherheitsanalyse

1. Grundlegende Isolationsmechanismen

1.1 Containerisierung – Isolation auf Kernel-Ebene

Container teilen sich den Kernel des Host-Systems. Die Trennung zwischen Containern und zwischen Container und Host wird ausschließlich durch Kernel-Features umgesetzt. Aus Sicht des Hosts sind Container lediglich Prozesse, die in einer eingeschränkten Sicht auf das System laufen. Die zentralen Mechanismen sind:

Namespaces schaffen pro Container eine separate Sicht auf bestimmte Kernel-Ressourcen. Linux kennt unter anderem den **(PID)**-Namespace (eigene Prozessbäume mit PID 1 im Container), **(NET)** (eigener Netzwerkstack mit Interfaces, Routing-Tabellen, iptables-Regeln), **(MNT)** (eigene Mountpoints), **(UTS)** (eigener Hostname), **(IPC)** (eigene System-V/POSIX-IPC), **(USER)** (eigenes UID/GID-Mapping, was es erlaubt, im Container Root zu sein, ohne es auf dem Host zu sein), **(CGROUP)** und **(TIME)**. Namespaces verhindern, dass ein Container Prozesse, Mounts oder Netzwerk anderer Container sieht – sie schränken aber nicht ein, *welche* Syscalls gegen den Kernel ausgeführt werden können.

Control Groups (cgroups v1/v2) sind orthogonal zu Namespaces und beschränken den *Ressourcenverbrauch*: CPU-Zeit, Speicher, Block-I/O, Netzwerk-Bandbreite, Anzahl Prozesse. Sie sind primär ein Schutz gegen Denial-of-Service durch noisy neighbors, kein Vertraulichkeits- oder Integritätsschutz.

Capabilities zerlegen das traditionelle Root-Privileg in rund 40 Einzelrechte (z. B. **(CAP_NET_ADMIN)**, **(CAP_SYS_ADMIN)**, **(CAP_SYS_MODULE)**). Container-Runtimes dropen standardmäßig die meisten Capabilities, sodass selbst ein Prozess, der innerhalb des Containers als UID 0 läuft, keinen Kernel laden oder beliebige Geräte mounten kann.

Seccomp-BPF filtert Syscalls auf Kernel-Ebene. Docker und Kubernetes liefern Default-Profile, die Hunderte von selten benötigten Syscalls (etwa **(keyctl)**, **(kexec_load)**, **(userfaultfd)**) blockieren. Da viele bekannte Container-Escapes Kernel-Bugs in genau solchen Syscalls ausnutzen, ist seccomp eine zentrale Verteidigungslinie.

Linux Security Modules (LSMs) wie AppArmor oder SELinux setzen Mandatory-Access-Control-Regeln durch und schränken Datei-/Dateisystemzugriffe orthogonal zu DAC ein.

1.2 Virtualisierung – Isolation durch Hypervisor

Eine virtuelle Maschine emuliert oder paravirtualisiert vollständige Hardware. Jede VM enthält ihren eigenen Kernel und Userspace. Zwischen Gast und Host steht der Hypervisor (Virtual Machine Monitor), der auf Hardwarevirtualisierungserweiterungen (Intel VT-x, AMD-V, EPT/NPT, IOMMU/VT-d) aufbaut.

Man unterscheidet **Type-1-Hypervisoren** (Bare-Metal: Xen, KVM, ESXi, Hyper-V), die direkt auf der Hardware laufen, und **Type-2-Hypervisoren** (Hosted: VirtualBox, VMware

Workstation), die als Anwendung auf einem Host-Betriebssystem laufen. Type-1 hat eine kleinere Trusted Computing Base und wird produktiv bevorzugt. Hardware-Erweiterungen sorgen dafür, dass privilegierte Instruktionen des Gasts gefangen und vom Hypervisor ausgewertet werden, ohne dass jeder Befehl interpretiert werden müsste.

Die Schnittstelle zwischen Gast und Host ist also nicht eine Syscall-Tabelle, sondern eine deutlich engere Menge: virtuelle CPU-Zustände, paravirtualisierte Geräte (virtio-net, virtio-blk), eventuell emulierte Geräte (legacy USB, Floppy, IDE). Diese Angriffsfläche ist um Größenordnungen kleiner als die Linux-Syscall-Oberfläche.

1.3 Bedrohungsmodelle und Vertrauensgrenzen

Der entscheidende Unterschied liegt in der Frage: *Welche Komponente muss bei einem Ausbruch versagen?*

Beim **Container** ist die Vertrauensgrenze der gemeinsame Host-Kernel. Jeder Syscall aus dem Container, der nicht von seccomp blockiert ist, läuft im Kontext des Host-Kernels. Eine ausnutzbare Kernel-Schwachstelle bedeutet potenziell einen Container-Escape. Vertraut wird also: dem Linux-Kernel inklusive aller geladenen Module, der Container-Runtime (runc, containerd, crun), dem Image (Lieferkette!) und der Konfiguration (Capabilities, seccomp, AppArmor).

Bei der **VM** ist die Vertrauensgrenze der Hypervisor und die emulierte/paravirtualisierte Hardware-Schnittstelle. Ein Kompromiss des Gast-Kernels betrifft erst einmal nur diese eine VM. Vertraut wird: dem Hypervisor, den Gerätemodellen (QEMU-Devices sind historisch ein Hotspot), Firmware (UEFI/SMM) und der Mikroarchitektur (Spectre, Meltdown, L1TF, MDS sind transversale Bedrohungen).

Faustregel: Container isolieren *Workloads*, VMs isolieren *Tenants*. Sobald gegenseitig misstrauische Mandanten auf derselben Hardware liegen sollen, sind klassische Container ohne zusätzliche Maßnahmen unzureichend.

2. Praktische Implementierung desselben Dienstes

Als Beispiel dient ein einfacher nginx-Webserver, der eine statische Seite ausliefert.

2.1 Containervariante (Docker)

Dockerfile:

```
dockerfile

FROM nginx:1.27-alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
USER nginx
```

Start mit sicheren Defaults:

bash

```
docker run -d \  
  --name web \  
  --read-only \  
  --cap-drop=ALL \  
  --cap-add=NET_BIND_SERVICE \  
  --security-opt=no-new-privileges \  
  --security-opt seccomp=/etc/docker/seccomp.json \  
  --tmpfs /var/cache/nginx --tmpfs /var/run \  
  -p 8080:80 \  
  web:latest
```

2.2 VM-Variante (Vagrant + KVM/libvirt)

Vagrantfile:

ruby

```
Vagrant.configure("2") do |config|  
  config.vm.box      = "generic/ubuntu2204"  
  config.vm.hostname = "web-vm"  
  config.vm.network "private_network", ip: "192.168.56.10"  
  
  config.vm.provider "libvirt" do |v|  
    v.memory = 1024  
    v.cpus   = 1  
  end  
  
  config.vm.provision "shell", inline: <<-SHELL  
    apt-get update -qq  
    apt-get install -y nginx  
    cp /vagrant/index.html /var/www/html/index.html  
    systemctl enable --now nginx  
  SHELL  
end
```

2.3 Vergleich der Isolationsgarantien

Ressourcenisolierung. Im Container regelt cgroups, wie viel CPU und RAM verbraucht werden dürfen, der zugrunde liegende Speicher ist aber derselbe physische Adressraum, vom Kernel verwaltet. Bei der VM bekommt der Gast einen abgegrenzten, durch EPT/NPT getrennten Adressraum; selbst ein bössartiger Gast-Kernel kann nicht direkt auf Speicherseiten anderer VMs zugreifen.

Netzwerkzugriff. Container hängen üblicherweise an einer virtuellen Bridge (`docker0`) mit veth-Paaren; iptables/nftables des Hosts setzen die Regeln. Ein Container im selben User-

Defined-Network kann Nachbarcontainer per DNS auflösen und direkt erreichen. Die VM bekommt eine virtuelle NIC (virtio-net), die an eine Linux-Bridge oder ein Open-vSwitch angeschlossen ist – semantisch näher an einem realen Netzwerkanschluss, der sich auch über VLANs sauber segmentieren lässt.

Systemzugriffsrechte. Container greifen direkt auf den Host-Kernel zu; jeder Syscall ist potenziell Angriffsfläche, wird aber durch Capabilities/seccomp/LSM gefiltert. Die VM ruft Syscalls gegen den eigenen Gast-Kernel auf; nur paravirtualisierte/emulierte Hardwarezugriffe queren die Hypervisor-Grenze.

3. Gezielte Fehlkonfigurationen und ihre Wirkung

3.1 Container

`docker run --privileged` deaktiviert nahezu alle Schutzmechanismen: Es werden alle Capabilities erteilt, AppArmor/SELinux umgangen, das seccomp-Profil entfernt, und alle Hostgeräte sind unter `/dev` erreichbar. Damit kann ein Container das Host-Dateisystem mounten, Kernel-Module laden oder über `cgroup release_agent` beliebigen Code als Root auf dem Host ausführen – ein dokumentierter, trivialer Escape-Pfad.

Das Mounten von `/var/run/docker.sock` in einen Container (häufig in CI-Pipelines) ist faktisch eine Root-Shell auf dem Host: Der Container kann `docker run -v /:/host --privileged ...` aufrufen und damit das gesamte Host-Dateisystem übernehmen.

`--cap-add SYS_ADMIN` allein gibt bereits weite Teile der Root-Rechte zurück (Mounting, Namespaces, BPF), ohne dass dies den Betreibenden meist bewusst ist.

`--pid=host`, `--network=host` oder `--ipc=host` heben den entsprechenden Namespace-Schutz auf. Mit `--pid=host` sieht der Container alle Host-Prozesse und kann sie unter den richtigen Capabilities auch beeinflussen.

Container als UID 0 ohne User-Namespaces zu betreiben bedeutet: Bei einem Escape (etwa über einen Kernel-Bug) ist der Angreifer sofort Host-Root. Aktive User-Namespaces hingegen mappen Container-Root auf eine unprivilegierte UID.

Das Deaktivieren des seccomp-Default-Profiles (`--security-opt seccomp=unconfined`) öffnet Hunderte Syscalls, von denen einige in der Vergangenheit für Eskalationen genutzt wurden (z. B. `userfaultfd` bei Heap-Spraying-Angriffen).

3.2 Virtuelle Maschinen

Eine VM mit gebrückter Schnittstelle direkt im Produktiv-VLAN unterläuft Netzwerksegmentierung und erlaubt Layer-2-Angriffe (ARP-Spoofing, Rogue DHCP) auf das Host-Netz. Korrekt wären VLAN-Trunks und tenant-spezifische Bridges.

Geteilte Ordner (VirtualBox Shared Folders, VirtFS/9p, virtiofs) etablieren einen Datenkanal über die Hypervisor-Grenze. Bugs in den entsprechenden Gerätemodellen waren mehrfach Angriffsvektor.

Aktivierte 3D-Beschleunigung (VirtualBox VBoxVGA, VMware SVGA) hat eine reichhaltige Geschichte ausnutzbarer Bugs, weil komplexe Grafik-Backends in C/C++ den Hypervisor-Adressraum berühren.

PCI-Passthrough ohne IOMMU/VT-d entzieht der Hardware die DMA-Schutzgrenze – ein bössartiger Gast kann per DMA in beliebige Hostspeicherbereiche schreiben.

Unverschlüsselte Live-Migration über das Datennetz exponiert Speicherinhalte und Schlüsselmaterial. Snapshots auf einem schwach geschützten Storage-Backend sind ein Offline-Angriffspfad.

Nested Virtualization vergrößert die TCB erheblich und sollte nur aktiviert werden, wenn benötigt.

4. Failure- und Escape-Szenarien im Vergleich

4.1 Container-Escapes

Container-Escapes treten in der Praxis wesentlich häufiger auf als VM-Escapes – sowohl wegen der größeren Angriffsfläche als auch wegen häufiger Fehlkonfiguration. Bekannte Kategorien:

Runtime-Bugs. CVE-2019-5736 (runc) erlaubte einem Container, durch geschicktes Überschreiben des `/proc/self/exe`-Symlinks beim nächsten `docker exec` das runc-Binary auf dem Host zu überschreiben und so Code als Host-Root auszuführen. CVE-2024-21626 ("Leaky Vessels") nutzte einen File-Descriptor-Leak in runc.

Kernel-Bugs. Da der Kernel geteilt wird, werden Lokalprivileg-Eskalationen zu Container-Escapes. Beispiele: Dirty Pipe (CVE-2022-0847), Dirty COW (CVE-2016-5195), CVE-2022-0185 (FS-Context-Heap-Overflow), zahlreiche Bugs in eBPF, Netfilter, io_uring. Ein gut konfiguriertes seccomp-Profil schließt einen erheblichen Teil dieser Angriffspfade.

Misconfiguration als Hauptursache. Privilegierte Container, gemountete Docker-Sockets, `hostPath`-Mounts in Kubernetes auf sensible Pfade (`/`, `/etc`, `/var/run`), `hostPID`, `hostNetwork` und unbeschränkte `securityContext.capabilities` sind in Audits häufig Auslöser realer Vorfälle. In Kubernetes verstärkt das Pod Security Admission diese Risiken, wenn `privileged`-Profil zugelassen wird.

Image-/Lieferketten-Angriffe. Schadhafte Basis-Images, kompromittierte Build-Pipelines oder typosquatting auf Registries führen zu Code-Ausführung im Container; ohne weitere Härtung kann das je nach Konfiguration den Host gefährden.

4.2 VM-Escapes

VM-Escapes sind selten, hochwertige Forschung und meist auf Bugs in Gerätemodellen zurückzuführen.

VENOM (CVE-2015-3456) im QEMU-Floppy-Controller war jahrelang in vielen Setups ausnutzbar, weil das Floppy-Device standardmäßig vorhanden war, selbst wenn keine Floppy

konfiguriert wurde.

Cloudburst, VMware-Escape-Ketten auf Pwn2Own (mehrfach) zeigen typischerweise eine Kette aus Bug im Grafik- oder USB-Subsystem plus Hypervisor-Speicherkorruption.

Mikroarchitektonische Seitenkanäle (Spectre, Meltdown, L1TF/Foreshadow, MDS, Zenbleed, Downfall) durchbrechen die Isolation auf Hardwareebene und betreffen sowohl Container als auch VMs. Mitigationen kosten Performance (KPTI, MDS-Buffer-Clear, retbleed-Mitigations).

Live-Migration und Storage-Backend sind operative Schwachstellen jenseits klassischer Escapes, die in realen Vorfällen genauso schaden.

4.3 Vergleich

VMs bieten eine deutlich härtere Trennung *gegenüber gegnerischen Mandanten*, sind aber nicht unverwundbar – insbesondere mikroarchitektonische Angriffe untergraben sie. Container sind schneller, dichter packbar und für vertrauenswürdige Workloads ausreichend isoliert, eskalieren bei Misskonfiguration aber rasch zu Host-Kompromiss. Hybridansätze wie **Kata Containers** (jeder Pod in einer leichtgewichtigen VM), **Firecracker** (von AWS für Lambda/Fargate) und **gVisor** (User-Space-Kernel-Reimplementation, der Syscalls abfängt) versuchen, beide Welten zu kombinieren.

5. Praktische Auswirkungen und Best Practices

5.1 Reale Deployment-Implicationen

In einer typischen Kubernetes-Plattform mit Workloads eines einzelnen Teams sind gut konfigurierte Container ein angemessenes Sicherheitsmodell – die Bedrohung sind primär kompromittierte Container-Workloads, nicht gegnerische Mandanten. Sobald jedoch echtes Multi-Tenancy ins Spiel kommt (öffentliche Function-as-a-Service-Plattformen, geteilte CI-Runner für externe Beiträge, Code-Execution-Sandboxes), ist eine VM-basierte oder Sandbox-basierte Isolierung der Standard. AWS Lambda und Fargate nutzen Firecracker; Google Cloud Run setzt auf gVisor; klassische IaaS-Anbieter geben dem Tenant ohnehin nur VMs.

Compliance-Regime (PCI-DSS, HIPAA, BSI-Grundschutz) verlangen häufig nachweisbare Trennungen, die mit reinen Containern schwer argumentierbar sind. In regulierten Umgebungen werden sensible Workloads üblicherweise in dedizierten VMs oder sogar dedizierten Hosts platziert.

5.2 Best Practices Container

Container sollten konsequent als unprivilegierte Nutzer laufen; in Kubernetes über `securityContext.runAsNonRoot: true` und `runAsUser` erzwungen. Das Root-Filesystem wird auf `readOnlyRootFilesystem: true` gesetzt, schreibbare Pfade als `emptyDir/tmpfs` gemountet. Capabilities werden vollständig gedroppt und nur einzelne benötigte (etwa `NET_BIND_SERVICE` für Ports unter 1024) zurückgegeben. Das Default-seccomp-Profil bleibt aktiv oder wird durch ein engeres ersetzt; `RuntimeDefault` ist Pflichtkonfiguration. AppArmor- oder SELinux-Profile

werden zugewiesen. User-Namespaces werden eingeschaltet, sodass Container-UID 0 außerhalb auf eine unprivilegierte UID gemappt ist – Podman nutzt das standardmäßig im Rootless-Mode.

Images stammen aus signierten Registries (Cosign, Sigstore), werden auf Schwachstellen gescannt (Trivy, Grype, Snyk) und sind möglichst minimal (Distroless, Chainguard, Alpine). `latest`-Tags werden vermieden, Pinning per Digest bevorzugt.

In Kubernetes wird Pod Security Admission auf `restricted` gesetzt, NetworkPolicies erzwingen Zero-Trust-Netz, Admission-Controller (Kyverno, OPA Gatekeeper) verhindern privilegierte Pods, hostPath-Mounts und unsignierte Images. Runtime-Detection (Falco, Tetragon) erkennt Anomalien wie unerwartete Shell-Aufrufe oder Container-Escapes in Echtzeit.

Für Workloads geringen Vertrauens werden Sandboxes wie gVisor oder Kata Containers eingesetzt – als RuntimeClass auswählbar.

5.3 Best Practices Virtuelle Maschinen

Hypervisoren werden zeitnah gepatcht und auf das Notwendige reduziert; ungenutzte Geräte (Floppy, parallel/serial, USB-Controller, virtuelle Soundkarten) werden im Gast deaktiviert, um die Geräteoberfläche zu verkleinern. PCI-Passthrough nur mit aktiviertem IOMMU. Nested Virtualization nur, wenn explizit benötigt.

Netzwerk wird über VLANs/VXLAN segmentiert, Management-Netz ist physisch oder per VLAN getrennt. Live-Migration läuft ausschließlich über verschlüsselte Kanäle (IPSec oder native TLS-Migration). Snapshots und Disk-Images werden verschlüsselt abgelegt (LUKS für Volumes, qcow2-encryption oder Storage-Backend-Encryption). Secure Boot und Measured Boot (TPM/vTPM) verankern die Integrität des Gast-Bootpfads.

Mikroarchitektonische Mitigationen (KPTI, IBRS/IBPB/STIBP, MDS-Clear) bleiben aktiv, auch wenn sie Performance kosten – das Abschalten lohnt nur in genau begründeten, einmandantenfähigen Szenarien.

5.4 Defense in Depth

Sicherheit entsteht aus geschichteten Maßnahmen. Container in VMs zu betreiben (etwa pro Mandant ein VM-Pool, darin Kubernetes), Workloads geringen Vertrauens in Sandbox-Runtimes zu kapseln, und kritische Daten auf dedizierte Hosts zu legen, kombiniert die Vorteile beider Modelle. Zentrales Logging, regelmäßige Audits der Konfigurationen (CIS Benchmarks für Docker, Kubernetes und gängige Hypervisoren) und Incident-Response-Pläne, die Container- und VM-Eskalationspfade kennen, runden das Bild ab.

Zusammenfassung in einem Satz

Container und VMs realisieren *graduell* unterschiedliche Isolationsstärken: Container teilen den Kernel und sind effizient, aber nur so sicher wie ihre Konfiguration und der darunterliegende

Kernel; VMs trennen vollständige Betriebssysteme über einen schmalen Hypervisor-Schnitt und sind die Standardantwort, wenn gegenseitig misstrauische Mandanten auf gemeinsamer Hardware laufen müssen – entscheidend für die reale Sicherheit ist in beiden Fällen weniger die Technologiewahl als die Disziplin in Konfiguration, Patching und Lieferkettenkontrolle.